Microsoft GraphRAG with an RDF Knowledge Graph - Part 3

Source: https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-3-328f85d7dab2
Published: 2024-08-24T16:17:24Z
PDF generated from reader view on 2026-06-27 16:02 UTC

Using SPARQL and the Knowledge Graph for RAG

Image 1: Ian Ormesher (https://medium.com/@ianormy?source=post_page---byline--328f85d7dab2----------)

5 min read

Aug 24, 2024

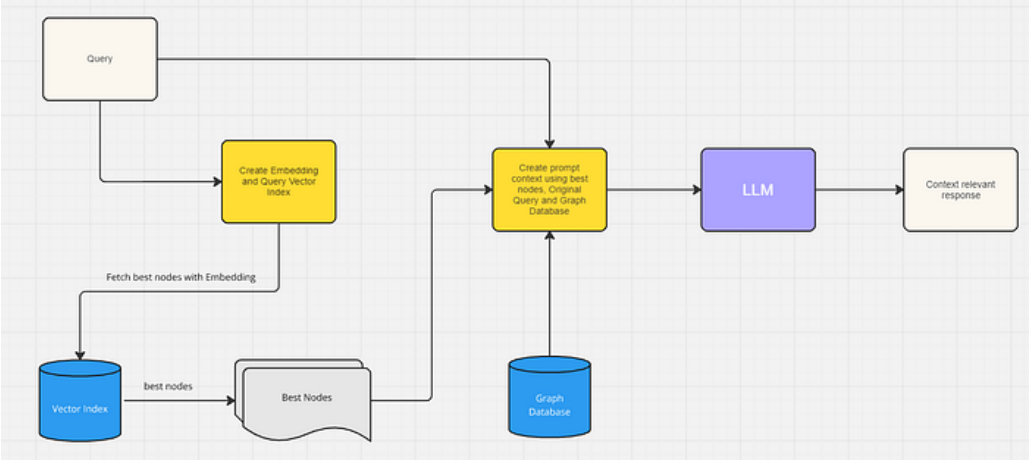

Image 2

Data flow for using the Knowledge Graph with RAG

Introduction

If you've been following this series then at this point you should have a populated Knowledge Graph in an RDF Store (I've been using GraphDB). The first two steps have been:

- Part 1 (https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-1-00a354afdb09) - Using a local LLM & Encoder to do Microsoft's GraphRAG
- Part 2 (https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-2-d8d291a39ed1) - Uploading the output from Microsoft's GraphRAG into an RDF Store

In this final part we're going to do the following:

- Encode our question into an Embedding Vector and do a search to find the 10 nearest `Entity` records to that vector.
- We then find the `Chunk` records that are associated with these `Entity` records and return the top 3
- We also find the `Community` records associated with these `Entity` records and return the top 3
- We also find the inside and outside relationships of these `Entity` records

- We also fetch the **description** of each `Entity`
- Having got all that information we feed that as context to our LLM together with our question

Encoding Our Question

To encode our question we will use our embedding endpoint provided by LM Server. This is an OpenAI endpoint. LM Server really helpfully provides code to show you how to connect to their server:

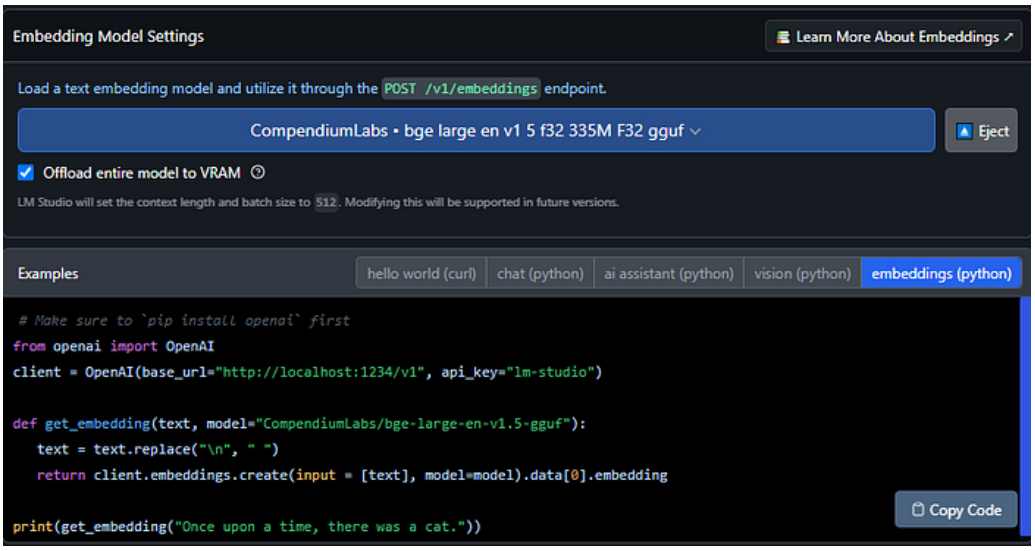

Image 3

LM Server Embeddings endpoint

`embedding_vector` will now be a vector of floating point values the same size are your encoding model. The encoding model I chose, `bge-large-en-v1.5-gguf` , creates a vector of size 1024. It is vitally important that you use the same encoding model for this step as you did in Part 1 (https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-1-00a354afdb09), otherwise your searches in vector space aren't going to work.

To search for our 10 nearest matching `Entity` records we are going to use the same Elasticsearch we created and used in Part 2 (https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-2-d8d291a39ed1). Assuming you have this up and running on your machine then you would do this as follows:

Now our `entity_list` contains a list of the top 10 `Entity` records` ids.

Get the top 3 Chunk records

Using this `entity_list` we are going to query our Knowledge Graph and get the top 3 `Chunk` records that are linked to these `Entity` records. For each identified `Chunk` we count the number of `Entity` records from our list that are linked and then order them by that count in a descending order:

Get the Top 3 Community Records

Again, using the `entity_list` we query our Knowledge Graph and get the corresponding `Community` records that are linked to these `Entity` records. We sort the `Community` records by rank and weight and take the top 3:

Get the Inside and Outside Relationships

We query our Knowledge Graph using the `entity_list` and the `related_to` relationship to find our inside and outside relationships:

And for the inside relationships:

Fetch the Entity Descriptions

We also fetch the `Entity` descriptions:

Feeding Context to the LLM

Having got all these DataFrames we want to convert each one into a format that we can then feed into our LLM. Here's an example of that conversion with the `entity_df` that we obtained before:

Create LangChain Response

Everything is now ready for us to setup an LLM chat with our local LM Server instance. LM Server once again provides us some helpful example code to connect to the server:

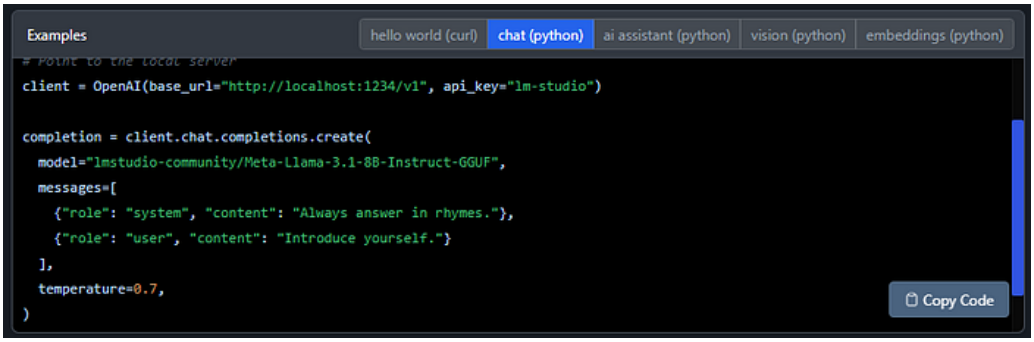

Image 4

LM Server OpenAI chat connection

We're actually going to be using LangChain (https://www.langchain.com/) to chain together our chat model and response:

First let's prompt with no context from our Knowledge Graph:

Here's the text I got back from my model:

> I think there may be some confusion here!

>

>

> In Charles Dickens' classic novel "A Christmas Carol", Bob Cratchit's wife is actually named Emily, not Belinda.

>

>

> Bob Cratchit is a kind and hardworking clerk who works for Ebenezer Scrooge. He is the father of six children: Peter, Belle (not Belinda), Tiny Tim, and three other unnamed children.

Interesting! I deliberately chose a question about a full name that I knew was not in the text, but that could be worked out by looking at the relationships. It's clearly failing here.

Now let's use the information we've obtained from our Knowledge Graph and put that into the context and ask the same question:

Here's the text I got back this time:

> According to the data provided, Belinda Cratchit is a daughter of Bob Cratchit. They are related as parent and child. Additionally, it is mentioned that Mrs. Cratchit (Bob's wife) assists Belinda with household tasks such as managing the cloth, indicating a close family relationship between them.

Nice! It clearly knows the book and has been able to work out the relationships from the information we provided from our Knowledge Graph.

Summary

I've enjoyed going on this journey of discovery with Microsoft's GraphRAG. I was able to run it on my local PC together with an LLM, Vector Embedding, Vector Index and RDF Store. I think the output responses I was able to get with my Chat show how much better this approach is than just simple RAG.

Next time I'm planning on blogging about Microsoft's GraphRAG Accelerator project which I was able to get up and running in the cloud and which also produces good results. Watch this space :-)